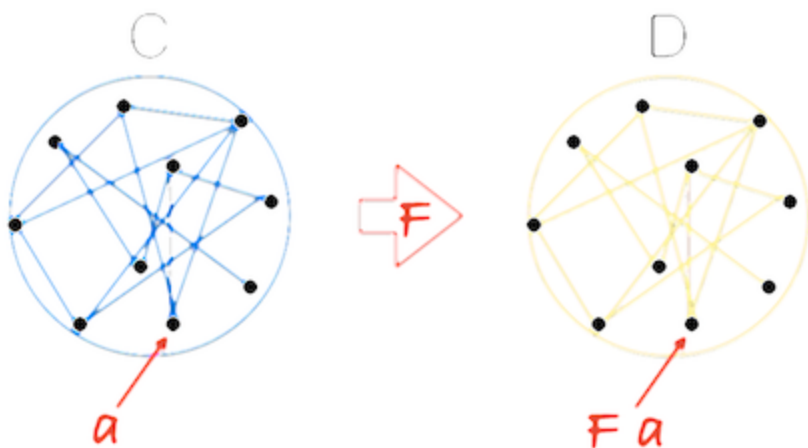


函数式编程之 Functor

什么是 Functor?



Functor 是实现了 `map` 函数，并遵守一些特定规则的容器类型 (mappable)

哪些数据结构能称为 Functor 呢?

数组

```
[ 2, 4, 6 ].map(x => x + 3)  
// => [ 5, 7, 9 ]
```

Promise

构造一个简单的 Functor

```
const Container = function(x) {  
  this.value = x  
}  
  
Container.of = function(x) {  
  return new Container(x)  
}  
  
Container.prototype.map = function(transform) {  
  return Container.of(transform(this.value))  
}
```

试着往 Container 装入一些值！

```
Container.of(2)  
//=> Container(2)
```

```
Container.of('hotdogs')  
//=> Container("hotdogs")
```

```
Container.of(Container.of({name: 'yoda'}))  
//=> Container(Container({name: "yoda" }))
```

使用 map 操作 Container 内的值!

```
Container.of(2)
  .map(value => value + 2)
//=> Container(4)
```

```
Container.of('hotdogs')
  .map(value => value.toUpperCase())
//=> Container("HOTDOGS")
```

```
Container.of('hotdogs')
  .map(value => value.toUpperCase())
  .map(value => value.length)
//=> Container(10)
```


绕来绕去的理由到底是什么？

组合

```
const compose = (f,g) => {  
  return x => f(g(x))  
}
```

实例

```
const toUpperCase = value => value.toUpperCase()  
const exclaim = value => `${value}!`  
const shout = compose(exclaim, toUpperCase)  
  
shout(`send in the clowns`)  
//=> "SEND IN THE CLOWNS!"
```

结合律

```
const toUpperCase = value => value.toUpperCase()
const exclaim = value => `${value}!`
const repeat = value => `${value}${value}${value}`
const repeatShout1 = compose(repeat, compose(exclaim, toUpperCase))
const repeatShout2 = compose(compose(repeat, exclaim), toUpperCase)

repeatShout1('send in the clowns')
//=> "SEND IN THE CLOWNS!SEND IN THE CLOWNS!SEND IN THE CLOWNS!"

repeatShout2('send in the clowns')
//=> "SEND IN THE CLOWNS!SEND IN THE CLOWNS!SEND IN THE CLOWNS!"
```

Curry 函数

```
const curry = fun => {  
  return arg1 => {  
    return arg2 => {  
      return fun(arg1, arg2)  
    }  
  }  
}
```

Curry 实例

```
// 获取 str 的前 n 个单词
const getNWord = (n, str) => {
  return str.split(' ')
    .slice(0, n)
    .join(' ')
}
const curriedGetNWord = curry(getNWord) // 步骤1
const get3Word = curriedGetNWord(3) // 步骤2
const shout3Word = compose(get3Word, compose(repeat, compo

shout3Word('send in the clowns')
//=> SEND IN THE
```

Curry + Functor

```
const MAP = R.curry((fn, F) => F.map(fn))

const double = value => value * 2
const mapDouble = MAP(double)

const addTwo = value => value + 2
const mapAddTwo = MAP(addTwo)

const addTwoAndDouble = compose(mapAddTwo, mapDouble)

addTwoAndDouble(Container.of(4))

//=> Container(10)
```

让我们回到 Promise!

Functor Laws:

1. Identity
2. Composition

Identity

```
const f = [1, 2, 3]
```

```
f.map(x => x)
```

```
// [1, 2, 3]
```

Composition

```
F.map(x => f(g(x)))  
// 等同于  
F.map(g).map(f)
```

Promise 满足吗?

```
/* Identity */
const p = Promise.resolve('1')
//=> Promise {[[PromiseStatus]]: "resolved", [[PromiseValue]]: "1"}

p.then(x => x)
//=> Promise {[[PromiseStatus]]: "resolved", [[PromiseValue]]: "1"}

/* Composition */
const g = value => value * 2
const f = value => value + 1

a.then(x => f(g(x)))
//=> Promise {[[PromiseStatus]]: "resolved", [[PromiseValue]]: 3}

a.then(g).then(f)
//=> Promise {[[PromiseStatus]]: "resolved", [[PromiseValue]]: 3}
```

问题一？

```
function addTwo(x) {  
  return x + 2  
}
```

```
const A = Container.of(2)  
const B = Container.of(addTwo)
```

applicative functor

```
Container.prototype.ap = otherContainer => {  
  return otherContainer.map(this.value);  
}
```

```
// 将 B 中方法 apply 到 A 中的值  
B.ap(A)
```

问题二？

```
Container.of(  
    Container.of(2)  
)
```

Monad

```
Container.prototype.map = transform => {  
  return Container.of(transform(this.value))  
}  
Container.prototype.join = () => this.value  
Container.prototype.flatMap = transform => {  
  return this.map(transform).join()  
}
```

```
const fs = require('fs');

const readFile = filename => {
  return new IO(function() {
    return fs.readFileSync(filename, 'utf-8')
  })
}

const print = str => {
  return new IO(function() {
    return str
  })
}

const tail = str => {
  return new IO(function() {
    return str[x.length - 1]
  })
}

// 读取
readFile('./user.txt')
  .flatMap(print)
```


换个名字?

```
// chain  
readFile('./user.txt')  
  .chain(print)  
  .chain(tail)
```

```
// then  
readFile('./user.txt')  
  .then(print)  
  .then(tail)
```

参考

What is a Functor?

JS 函数式编程指南

函数式编程入门教程

Monads in JavaScript

Functors & Categories



